

ADR-001: Verwendung einer Layered Architecture

Status: Akzeptiert

Datum: 08.05.2026

Kontext

Für die Entwicklung der Anwendung wird eine Softwarearchitektur benötigt, die eine klare Trennung von Verantwortlichkeiten ermöglicht, leicht verständlich ist und eine langfristige Wartbarkeit gewährleistet.

Das System umfasst verschiedene fachliche Funktionen, Datenpersistenz sowie eine API-Schnittstelle. Ohne eine strukturierte Architektur würden Geschäftslogik und Datenzugriffe eng miteinander gekoppelt werden, was die Wartung, Erweiterung und Testbarkeit erschwert.

Entscheidung

Die Anwendung wird nach dem Prinzip der Layered Architecture (Schichtenarchitektur) aufgebaut.

Die Architektur wird in folgende Schichten unterteilt:

Controller Layer

- Bereitstellung der API-Endpunkt
- Annahme und Rückgabe der Daten
- Error Handling per http-Status

Business Layer (Service Layer)

- Implementierung der Geschäftslogik
- Koordination von Anwendungsfällen
- Durchsetzung fachlicher Regeln
- Wird um Domain-Objects (Entitys / Value Objects) ergänzt

Persistence Layer (Repository Layer)

- Datenzugriff
 - Kommunikation mit Datenbank
 - Kapselung der Persistenz-Logik

Konsequenzen

Vorteile

- Klare Trennung von Verantwortlichkeiten (Separation of Concerns)
- Verbesserte Wartbarkeit und Lesbarkeit des Codes
- Höhere Testbarkeit durch isolierte Schichten
- Austauschbarkeit einzelner Komponenten ohne Auswirkungen auf andere Bereiche
- Gute Unterstützung durch Frameworks wie Spring Boot
- Bekannte und weit verbreitete Architektur mit geringer Einarbeitungszeit

Nachteile

- Zusätzlicher Boilerplate-Code zwischen den Schichten
- Höherer Implementierungsaufwand bei kleinen Anwendungen
- Daten müssen oftmals über mehrere Schichten weitergereicht werden
- Gefahr von unnötigen Abhängigkeiten bei fehlender Architekturdziplin

Begründung

Die Layered Architecture wurde gewählt, da sie eine einfache und bewährte Struktur für die Entwicklung geschäftsorientierter Anwendungen bietet. Sie unterstützt die Trennung fachlicher und technischer Verantwortlichkeiten und erleichtert sowohl die Wartung als auch die Erweiterung des Systems. Aufgrund der erwarteten zukünftigen Erweiterungen und der Nutzung von Spring Boot überwiegen die Vorteile dieser Architektur die entstehenden Zusatzaufwände.

Verworfen Alternativen

- Monolith ohne Schichten
 - Einfacher Initialaufbau
 - Führt langfristig zu starker Kopplung und schlechter Wartbarkeit
- Hexagonal Architecture
 - Sehr gute Entkopplung von Infrastruktur und Fachlogik
 - Für den Projektumfang zu komplex
 - Höherer Implementierungs- und Dokumentationsaufwand
- Microservice-Architektur
 - Gute Skalierbarkeit
 - Nicht erforderlich für die aktuelle Projektgröße
 - Zusätzliche Komplexität hinsichtlich Deployment und Kommunikation